

DDoS Detection and Mitigation System

Python Network Security Engineering Project

Raliegth Barnett | Cybersecurity & Networking Portfolio

Portfolio Case Study - Detailed Project Oversight and Completion Record

All offensive-security activities described in this document were performed in authorized, isolated academic laboratory environments.

1. Project Overview

A real-time defensive monitoring tool that captures TCP SYN traffic, learns a baseline, detects anomalous packet rates, visualizes activity, logs events, sends alerts, and can block the highest-volume source in an authorized lab environment. The attack demonstration and detector video are treated as one integrated project.

Project Field	Details
Role and Responsibility	Designer, Python developer, test operator, and technical documenter.
Environment and Tools	Kali Linux / Linux host, Python 3, Scapy, Matplotlib, iptables, SMTP, hping3, isolated lab network.
Completion Status	Completed - documented, tested, and included in the master portfolio.
Portfolio Classification	Academic / personal technical project with recruiter-facing case-study documentation.

2. Project Objectives and Success Criteria

- Capture TCP SYN packets arriving on a selected interface and service port.
- Establish a short learning period and calculate an adaptive baseline threshold.
- Classify traffic as normal or a potential denial-of-service event.
- Display a real-time traffic graph and top source IP addresses.
- Generate event logs, email notifications, and optional iptables blocking.
- Demonstrate the system through a controlled SYN-flood test and recorded video.

3. Project Oversight and Execution Process

3.1 Planning and Scope

Defined the authorized environment, project goal, tools, expected outputs, and safety limitations.

3.2 Implementation

Executed the technical tasks and maintained notes, commands, screenshots, and configuration evidence.

3.3 Validation and Testing

Compared actual output with objectives, confirmed repeatable behavior, and collected proof of completion.

3.4 Analysis and Reporting

Converted technical evidence into findings, defensive interpretation, limitations, and recommendations.

3.5 Portfolio Finalization

Prepared a recruiter-friendly summary, detailed case study, screenshots, source report, and public-release guidance.

4. Technical Methodology

1. Reviewed the network-monitoring requirement and defined observable indicators based on SYN packet rate.
2. Implemented packet capture with Scapy and a background sniffer thread.
3. Created a one-second aggregation loop, rolling history buffer, and baseline-learning logic.
4. Added visualization, logging, alert cooldown, top-source reporting, and automated blocking.
5. Generated controlled test traffic only inside the authorized lab and documented the response.
6. Sanitized credentials from the portfolio version of the source code.

5. Deliverables and Completion Evidence

Deliverable	Status	Evidence
Technical implementation / lab execution	Complete	Commands, configurations, code, or forensic workflow documented.
Testing and validation	Complete	Results and screenshots included below and in the original report.
Detailed project documentation	Complete	This case-study document and README.
Source/original evidence	Complete	Original report or source files stored in this project folder.
Recruiter-ready description	Complete	Summary and skills section included.
Public publishing review	Required before posting	Remove credentials, personal identifiers, confidential case data, and unnecessary target details.

6. Results and Outcomes

- The detector transitions from learning mode to active monitoring and compares each interval with a learned threshold.
- During controlled high-rate SYN traffic, the console changes to an attack state, records the rate, identifies top sources, and invokes configured response actions.
- The demo video provides visual evidence of the end-to-end workflow.

7. Challenges, Decisions, and Lessons Learned

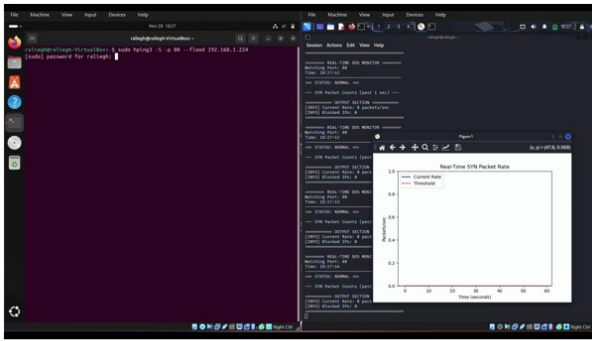
- Avoiding false positives when baseline traffic is very low.
- Coordinating packet capture, graph updates, and event processing without blocking the interface.
- Protecting email credentials and separating secrets from source code.

8. Screenshots and Supporting Evidence

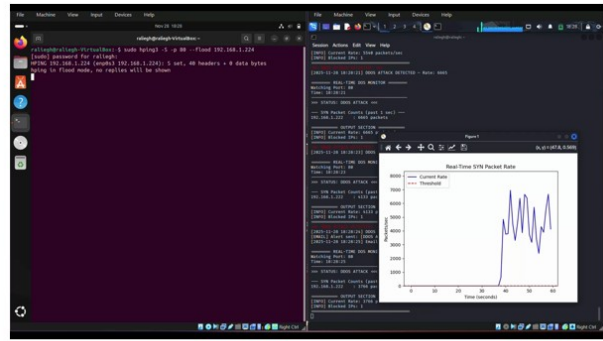
DDoS Detection and Mitigation Workflow



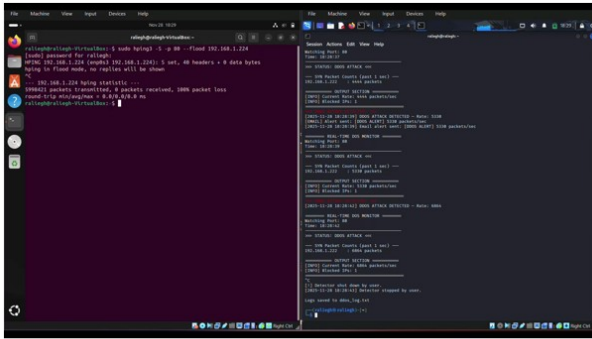
System architecture showing packet capture, baseline learning, detection, alerting, visualization, and response.



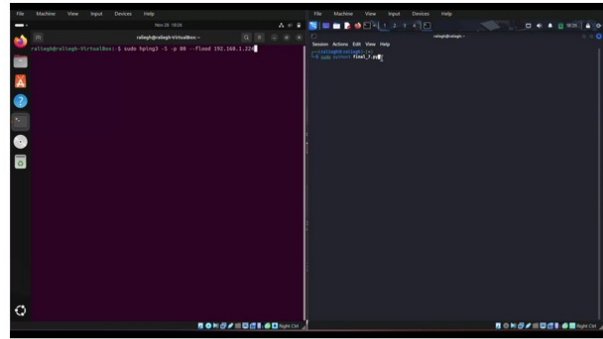
frame_100.png



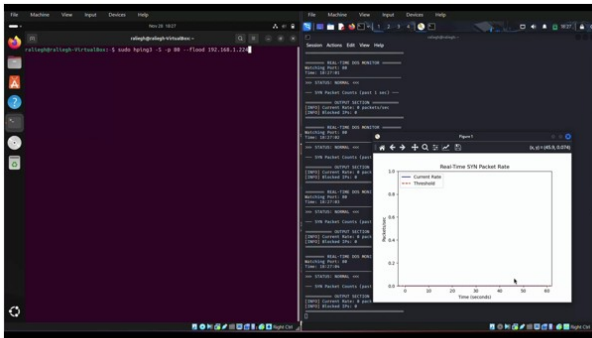
frame_140.png



frame_180.png



frame_20.png



frame_60.png

Contact sheet from the demonstration video showing the detector and controlled test workflow.

9. Skills Demonstrated

Python, Scapy, TCP/IP, threading, Matplotlib, Linux, iptables, logging, SMTP alerts, defensive testing, technical documentation.

10. Recommendations and Future Improvements

- Use robust statistics such as median and standard deviation rather than a simple average multiplier.
- Add per-source thresholds, rate limiting, allowlists, and block expiration.
- Store events in SQLite and provide a web dashboard.
- Add unit tests, configuration files, and environment-variable based secret management.

11. Portfolio and Resume Presentation

DDoS Detection and Mitigation System - A real-time defensive monitoring tool that captures TCP SYN traffic, learns a baseline, detects anomalous packet rates, visualizes activity, logs events, sends alerts, and can block the highest-volume source in an authorized lab environment. The attack demonstration and detector video are treated as one integrated project.

Recommended repository contents: README.md, Documentation/, Evidence/, Source/ or Original_Report/, and a clear authorization/privacy notice.

12. Completion Statement

This project is considered complete for portfolio use because the scope, implementation process, testing evidence, results, skills, limitations, and future improvements are documented. Before public release, sensitive data must be reviewed and removed.